為何需要平行處理

單核頻率不能無限提高，原因包括 power density、clock distribution、instruction-level parallelism（ILP，指令層平行）有限與 memory wall。多核把工作分給多個執行單元，目標是提高 throughput 或降低 large problem 的 elapsed time。

多核處理器把多個 cores 放在同一 die；multiprocessor 可以跨 sockets；cluster 用多台 computers 與 network。三者都可稱為平行系統，但 communication latency、coherence 與 failure boundary 不同。

本章的 speedup 指同一問題或同一服務需求下的 elapsed-time 比值。沒有 workload 定義的「N 倍快」不具工程意義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-stu

Parallelism 的種類

Job-level/process-level parallelism 讓多個 independent jobs 同時執行，例如數個 web requests。這類 throughput scaling 不要求每題被拆成多份。

Parallel processing program 讓一個 job 使用多個 workers。Newton iteration、matrix multiplication 與 image processing 可能屬於此類，但程式的 dependencies 決定可平行比例。

Instruction-level parallelism 由 pipeline/superscalar 在單一 instruction stream 內找 independent operations；data-level parallelism 對大量元素做相同 operation；task-level parallelism 對不同功能或 requests 分工。

Lin Cing
https://github.com/xinhi6716/computer-science-self-stu

7.1 Power-aware parallel design

若降低 supply voltage 與 frequency，單核 energy/operation 常下降，但效能也下降。把相同 power budget 分給多個較慢 cores，可能提高 aggregate performance per watt，前提是程式能用到這些 cores。

Parallelism 也提供 availability：一個 processor 失效時，replicated service 可以由其他 nodes 繼續，但這需要 redundancy、failure detection 與 recovery，不等於只要有多核就會自動可靠。

Scalable software 必須能在不同 processor counts、cache capacities 與 network topologies 上維持正確與合理效率。

「小核比較省電」只在指定電壓/頻率與 workload 下有量化意義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.2 Amdahl 的固定工作模型

令固定工作的單執行緒 baseline 為 T_0。若其中比例 f 為可平行部分，$(1-f)$ 為序列部分，且 N 個 workers 完美分擔平行部分、沒有額外成本：

$$T_N = T_0(1-f) + \frac{T_0 f}{N}.$$

這個式子把「序列部分不縮小」直接寫進時間分解。Speedup 是 baseline time 除以新 elapsed time：

$$S_N = \frac{T_0}{T_N} = \frac{1}{1-f+f/N}.$$

Efficiency 定義為 $E_N = S_N/N$，單位不是 occupancy。若 $S_N = 4.70588, N = 8$，則 $E_N = 0.588235 = 58.8235\%$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Amdahl、強弱 scaling 與負載上限

固定工作	速度上限	反解 fraction
$T_N = T_{seq} + T_{par}/N$ $T_0 = T_{seq} + T_{par}$	$S_N = T_0/T_N$ $S_\infty = 1/(1-f)$	$f \geq (1-1/S)/(1-1/N)$ $S \leq N$；等號需 $f=1$

N=100、目標 S=90 → f≥890/891≈0.998878；serial≤1/891≈0.1122%。

例：10 個 scalar + 1000 個 matrix additions，N=100 → T=10+10=20；
matrix 增為 10000 個 additions → T=10+100=110，S=10010/110≈91。

強 scaling 固定問題；弱 scaling 問題隨處理器成長。兩者 baseline 與結論不可互換。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

當 $N \to \infty$：

$$S_\infty = \frac{1}{1-f}.$$

這個上界只有在 workload、baseline 時間比例與成本定義固定時成立。它是 asymptote，不是某一台機器的 measured speedup。

從目標 speedup 反解

若已知 N 與目標 $S < N$：

$$f \geq \frac{N(S-1)}{S(N-1)}.$$

例如 $N = 100, S = 90$，必要 $f \geq 8900/8910 = 0.998878$，序列比例不得超過約 0.1122%。把這個門檻誤解成「90% 可平行」會得到完全不同的答案。

模型沒有包含 synchronization、load imbalance、launch 或 shared-bandwidth contention。實際時間可以寫成

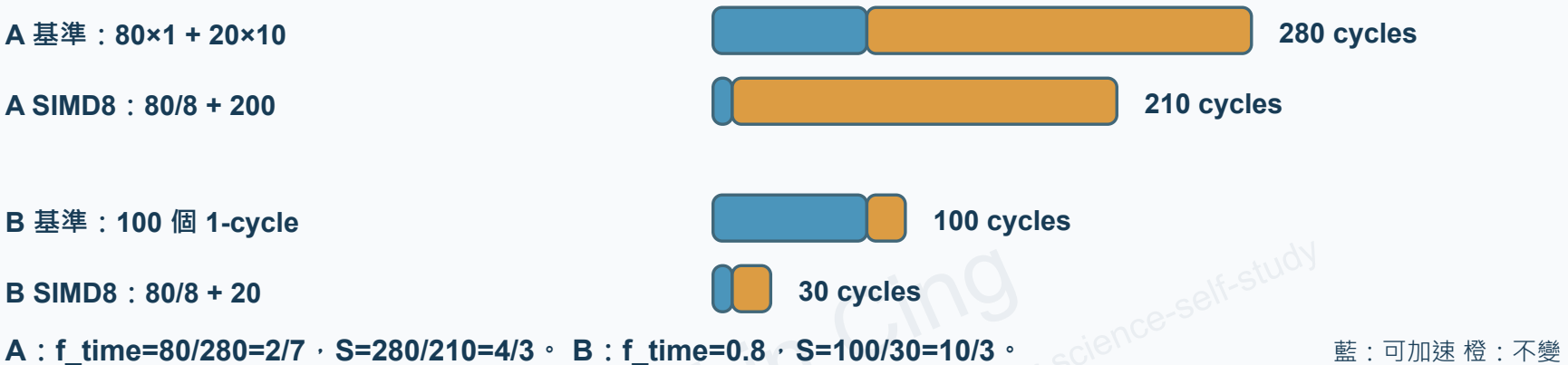
$$T_{N,\text{observed}} = T_N + T_{\text{critical}} + T_{\text{sync}} + T_{\text{contention}},$$

因此 measured speedup 通常低於 ideal Amdahl 模型。只有先驗證這些成本近似可忽略，才能把差值歸因於單一原因。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

操作比例不等於時間比例

Amdahl 使用基準時間，不是 operation share



先看100個操作中的80個一cycle操作可向量化，另20個各花10 cycles。基準時間為$80 + 200 = 280$ cycles。8倍向量化後為$10 + 200 = 210$ cycles，故$S = 4/3$，可加速時間比例只有$80/280 = 2/7$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

操作比例不等於時間比例 (續)

模型	基準可加速時間	基準其餘時間	8倍加速後總時間	整體加速
不等成本A	80	200	10+200=210	280/210=1.3333
等成本B，100個操作各1 cycle	80	20	10+20=30	100/30=3.3333

相同80%操作比例可以對應不同時間比例。只有等成本或另外量到80%基準時間時，才可將0.8直接代入Amdahl。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strong scaling、weak scaling 與效率

Strong scaling 固定 problem size W，增加 processors P，問同一工作最快能多快完成。若理想時間是 T_1/P，則

$$S_{\text{strong}}(P) = \frac{T_1}{T_P}, \qquad E(P) = \frac{S_{\text{strong}}(P)}{P}.$$

Weak scaling 讓總 problem size 從 W 成長為 PW，使每 processor 的工作量固定。弱擴展效率定義為

$$E_{\text{weak}}(P) = \frac{T_1(W)}{T_P(PW)}.$$

理想弱擴展保持相同完成時間，因此效率為1；此時間比不是固定問題的 speedup，也不應再除一次 P。若另外假設同一問題的單核成本隨工作量線性，$T_1(PW) = PT_1(W)$，才可寫

$$S_{\text{scaled}}(P) = \frac{T_1(PW)}{T_P(PW)} = \frac{PT_1(W)}{T_P(PW)} = PE_{\text{weak}}(P).$$

若缺少較大問題的單核時間或線性成本假設，不能僅從兩個不同規模的時間推出這個同規模 speedup。若資料量 M 固定，strong scaling 的每處理器記憶體約 M/P；若 weak scaling 讓 M 隨 P 成長，每處理器記憶體約維持 M。

同一公式的兩個完整案例

假設每次 addition 成本為 t，序列工作為 10 次 additions。

問題	$P = 10$	$P = 100$
small：10 serial + 100 parallel	$T = 10t + 100t/10 = 20t$，$S = 110/20 = 5.5$	$T = 10t + 100t/100 = 11t$，$S = 110/11 = 10$
large：10 serial + 10,000 parallel	$T = 10t + 10,000t/10 = 1010t$，$S = 10010/1010 \approx 9.91$	$T = 10t + 10,000t/100 = 110t$，$S = 10010/110 = 91$

小問題固定時，100 cores 只能得到 10 倍，因序列成本占比變大。問題隨處理器成長後，100 cores 可接近 100 倍。這是 Amdahl 與弱 scaling 能夠同時成立的原因：工作與分母一起改變。

報告至少記錄 problem size、per-core work、worker count、baseline、communication bytes、memory footprint、clock/power policy 與 measured wall time。只寫「99% 效率」而沒有 growth rule，無法重現。

Load balance 與 parallel critical path

令總 parallel work 為 W、最重 worker 的工作量為 $W_{\max}$、processor 數為 P、固定 overhead 為 C。其他 workers 平均分擔剩餘工作時，parallel phase 的 completion time 下界是

$$T_{\text{parallel}} = \max\left(W_{\max}, \frac{W - W_{\max}}{P - 1}\right),$$

$$T_{\text{total}} = T_{\text{parallel}} + C.$$

完整算例

令 $W = 10{,}000$、$C = 10$、$P = 100$。若一個 worker 分到 2%，即 200 units，其他 99 workers 共 9,800 units：

$$T = \max\left(200, \frac{9800}{99}\right) + 10 = \max(200, 98.99) + 10 = 210.$$

Sequential baseline 是 $10,000 + 10 = 10,010$：

$$S = 10{,}010/210 \approx 47.67, \qquad E = 47.67/100 \approx 47.7\%.$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Load balance 與 parallel critical path （續）

若重 worker 分到 5%（500 units）：

$$T = \max\left(500, \frac{9500}{99}\right) + 10 = 500 + 10 = 510,$$

$$S = 10{,}010/510 \approx 19.63, \qquad E \approx 19.6\%.$$

一般 worker 的平均工作量不能預測完成時間；最慢 worker 與 dependency chain 都可能成為 critical path。Work stealing 可動態搬移 tasks，但每次 steal 需要 queue/atomic metadata，搬移資料又增加 cache 與 network traffic。Task 太細時，coordination 成本可能超過負載改善。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.3 Shared-memory multiprocessors

SMP（shared-memory multiprocessor，共享記憶體多處理器）給所有 processors 一個 common physical address space。Programs 用 shared loads/stores 溝通，硬體以 cache coherence 維持同一位置的 visible order。

UMA（uniform memory access）模型中，各 processor 到 memory 的 access time 近乎相同。NUMA（nonuniform memory access）中，local memory 比 remote memory 快；program page placement 與 data affinity 會改變效能。

NUMA 系統仍共用同一個 physical address space。NUMA 描述 memory latency 與 bandwidth 的拓樸差異，不是彼此分離的 address spaces。

Directory-based coherence 常用於較大的 SMP/NUMA，snooping 在 broadcast domain 小時較直接。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shared-memory synchronization

同步必須同時處理三件事：

1. **Atomicity**：read-modify-write 不可被拆成其他人可觀察的中間狀態。
2. **Ordering**：哪些 memory operations 必須排在另一個 operation 前後。
3. **Visibility**：建立 ordering 後，consumer 讀到 producer 寫入的值。

Coherence 只保證同一位置的 cached copies 與 write serialization。跨不同位置的 publication 仍需要 memory consistency model 所定義的 synchronization。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shared-memory synchronization （續）

共享變數 publication：atomicity、ordering 與 visibility

Writer thread

data = 42

release store ready = 1

Reader thread

acquire load ready

if value == 1: read data

正確關係：release 先排序 data store 與 ready store；acquire 讀到該 release 值後，才建立 synchronizes-with；reader 的後續 data load 因此看到 42。

獨立 relaxed 版本：data / ready 皆 atomic；ready=1 後仍可讀到 data=0。

Plain non-atomic concurrent accesses 是 data race；acquire/release 只排序規格涵蓋的衝突存取。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Publication worked example

```
int data = 0;                  // shared object
atomic_int ready = 0;

// Writer
data = 42;
atomic_store_explicit(&ready, 1, memory_order_release);

// Reader
while (!atomic_load_explicit(&ready, memory_order_acquire))
    yield();
int observed = data;
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

當 reader 的 acquire load **實際讀到** writer 的 release store 所寫入的 `1` 時，release 與 acquire 建立 synchronizes-with，再透過 program order 建立跨 thread happens-before。`data=42` 必須先於 `data` load 可見，所以 `observed=42`。

Shared-memory synchronization （續）

作為可合法討論讀值的錯誤版本，改令 `data` 與 `ready` 都是 atomic，兩者的 store/load 全採 `memory_order_relaxed`。Writer 依序寫 data=42、ready=1；reader 讀到 ready=1 後仍可讀到 data=0，因為不同 atomic objects 之間沒有 synchronizes-with，coherence 只約束各自的 modification order。若兩者改成 plain non-atomic 且沒有其他同步，則形成 C/C++ data race、行為未定義；此時不能宣稱只會得到0或42。

同步只排除它涵蓋的衝突 accesses。若程式另有第三個 thread 在 lock 外修改同一資料，該 race 不會被修好。`atomic` 也不自動讓多個位置的 transaction 原子化。

Spinlock 以 atomic read-modify-write 嘗試取得 lock；contended spin 消耗 interconnect 與 power。Blocking mutex 將 waiting thread 交給 OS，代價是 context switch。Barrier 要所有 participating threads 到達後才釋放；tree barrier 以 $O(\log P)$ 深度換取較小 fan-out，但節點同步與喚醒成本仍存在。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.4 Clusters 與 message passing

Cluster 由多台 commodity nodes 透過 network 連接，每個 node 有自己的 OS 與 physical address space。Communication 由 explicit send/receive 或 RPC/queue 完成，不使用 ordinary shared load/store。

Message passing 讓 sender/receiver ordering 較明確，但需要 serialize data、copy 到 network buffer、route、receive 與 possibly ACK。Failure isolation 較好，recovery 仍要 application framework。

MPP（massively parallel processor）類歷史上使用 specialized network；modern clusters 以 commodity switch 與 software stack 為主，取決於 latency/bandwidth/price。

Cluster 不是免費的 shared memory；distributed shared-memory layer 也建立在 message passing 之上。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.5 Hardware multithreading

Hardware thread 是 CPU 可快速切換的 execution state，分開 PC、register file 與一些 control state；functional units、caches 與 memory 可以共享。它是處理 memory/instruction latency 的 microarchitecture 方法。

Fine-grained multithreading 每 cycle 輪流選 thread，ready thread 也可能被別 thread 分享 issue slots；適合隱藏長短 stalls，但單 thread latency 可能增加。

Coarse-grained multithreading 在 long stall 時才切換 thread；單 thread normal execution 較少被打斷，但可能無法隱藏短 dependency。

Simultaneous multithreading（SMT）在同 cycle 的 superscalar issue slots 中混合多 threads，對 ILP 不足有幫助；代價是 register storage、scheduler、cache pressure。

每個 core 執行兩個 threads 不代表 throughput 加倍；實際結果取決於 issue width、cache 與 workload。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.6 SISD、MIMD、SIMD、SPMD 與 vector

SISD 是一個 instruction stream 作用於一個 data stream；MIMD 是多 processors 各有 independent instruction/data streams。SIMD 讓 one instruction 同時處理多個 data elements；SPMD 是 programming model，由多個 threads 執行同一 program、不同 data。

SPMD 可在 MIMD cluster 或 SIMT GPU 上實作。SPMD 不是固定硬體 classification，不能和 SIMD 當成同一層概念。

SIMD 擴充適合 regular contiguous data：array add、image filter、packed integer。Control flow 複雜或 data dependencies 不規則時，lane utilization 下降。

先列 software model 與 hardware execution model，再談 classification。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Vector architecture

Vector ISA 使用 vector registers 與 vector length。對 L 個 elements，一條 vector instruction 描述整個 vector operation；硬體以 lanes、chaining 或 pipelining 執行。

Vector code 通常用 vector length 表示 semantic，讓編譯器與硬體處理 tails 和不同 hardware widths。Multimedia SIMD 常固定 128/256 bits，software 要自行處理 padding/tail 與 narrower hardware。

令每個element做k operations，硬體有L lanes且每lane每cycle可完成一個element，frequency為f cycles/s、有效利用率為η。此模型的throughput為

$$\mathrm{OP/s} = kLf\eta$$

量綱為operations/element × lanes × elements/(lane • cycle) × cycles/s。若每lane initiation interval是q cycles/element，則再除以q。另一個等價描述是每條vector instruction處理W elements、每cycle可啟動r條，則上限為$kWrf$；vector length W不是physical lane數，兩者不可重複相乘。

Vector 的增益來自 amortize control/fetch 與 regular lane usage，不是 lane 數必然等於 real speedup。

7.7 GPU 的定位

GPU 有大量較 simple cores、高 memory bandwidth 與 many warps。CPU 偏向少量 latency-sensitive threads；GPU 以 massive multithreading 隱藏 memory latency，適合 high arithmetic/data parallelism。

GPU 的效能來自 throughput，而非 single-thread latency。Host-device transfer、kernel launch、divergence、coalescing 與 occupancy 都會影響 end-to-end time。

本節只建立比較框架；SIMT warp、CUDA memory spaces 與應用 mapping 在 Appendix A 展開。

不要把「GPU core 數量較多」直接解讀為單一 scalar program 較快。

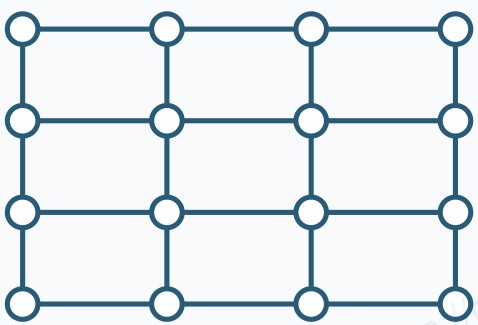
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.8 Multiprocessor network topologies

把 network 寫成 graph $G = (V, E)$。Node 可代表 processor-memory endpoint，link 可有 bandwidth 與 hop latency。以下都把 full-duplex physical link 計為一條 edge；若報 bisection bandwidth，必須先說每次 link 是只計單向頻寬或雙向加總。

7.8(a) 4×4 mesh

4×4 mesh：24 條 local links，無 wrap



Mesh 指標

horizontal = 4×(4−1) = 12

vertical = 4×(4−1) = 12

edges = 24；average degree = 48/16 = 3

corners degree 2；interior degree 4

diameter = (4−1)+(4−1) = 6

bisection links = 4

沒有第 4 欄→第 0 欄或第 4 列→第 0 列的 link，所以不是 torus。

四列各有 $4 - 1 = 3$ 條 horizontal links，合計 12；四欄各有 $4 - 1 = 3$ 條 vertical links，合計 12：

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.8 Multiprocessor network topologies （續）

$$|E_{\text{mesh}}| = 12 + 12 = 24.$$

Degree 總和是 $2|E| = 48$，所以平均 degree 是 $48/16 = 3$。角落 degree=2、邊上 degree=3、內部 degree=4。任兩點最遠需水平 3 hops 加垂直 3 hops：

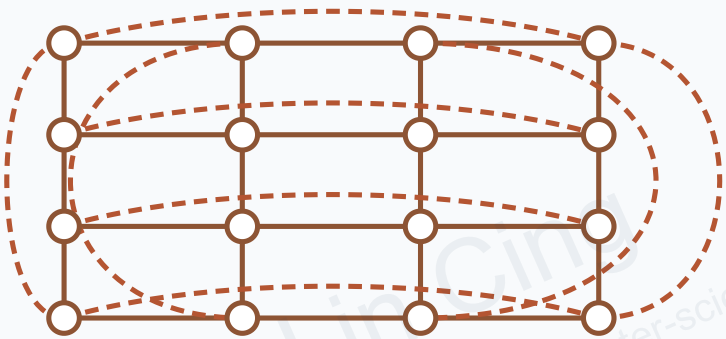
$$D_{\text{mesh}} = 3 + 3 = 6.$$

把四欄切成左 2 欄與右 2 欄時，跨越切面的只有 4 條 horizontal links；其他 balanced cuts 不會更少，因此 bisection links=4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.8(b) 4×4 torus

4×4 torus：24 local + 8 dashed wrap = 32 links



Torus 指標

degree = 4 for every node

wrap links = 4 horizontal + 4 vertical

total edges = 24 + 8 = 32

diameter = floor(4/2)×2 = 4

bisection links = 8

H0–H3：(0,r)↔(3,r)。 V0–V3：(c,0)↔(c,3)。 虛線就是 8 條 wrap links。

同一列與同一欄都形成長度 4 的 cycle；四個 mesh corner 在 torus 中 degree 仍是 4。

Torus 保留 mesh 的 24 條 local links，再把每一列的右端連回左端（4 條），並把每一欄的下端連回上端（4 條）：

$$|E_{\text{torus}}| = 24 + 4 + 4 = 32.$$

每個 node 都有左右上下四條 links，所以 degree=4。任兩 node 在每一維最多繞行 $\lfloor 4/2 \rfloor = 2$ hops：

$$D_{\text{torus}} = 2 + 2 = 4.$$

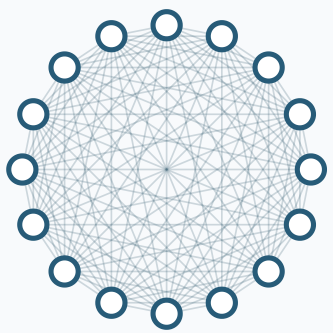
把 columns 0–1 與 columns 2–3 分開，跨切面是 4 條 local horizontal links 加上 4 條 wrap links，共 8。程式對全部 balanced cuts 取最小，得到 bisection links=8。

圖中的虛線不是裝飾：$H0$–$H3$ 是四列各自的 $(0, r) \leftrightarrow (3, r)$，$V0$–$V3$ 是四欄各自的 $(c, 0) \leftrightarrow (c, 3)$。少了這 8 條，圖就只是 mesh，degree 與 diameter 都不再符合 torus。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.8(c) K16 與 crossbar 的模型差異

K16：16 個 endpoints 兩兩直接相連



K16 指標

edges = N(N−1)/2 = 16×15/2 = 120

degree = 15

diameter = 1

bisection = (N/2)² = 8² = 64 links

單向 cut capacity = 64×link rate

K16 的 endpoint graph 不是 central crossbar；crossbar 是另一種 switch 資源模型。

若每條 link 是 full-duplex，雙向 cut 合計為 2×64×單向 rate；報告先固定加總慣例。

K_{16} 是完全圖：

$$|E| = \frac{16(15)}{2} = 120, \quad \deg(v) = 15, \quad D = 1.$$

任一 balanced cut 有 $8 \times 8 = 64$ 條跨越 links；若 link 的單向頻寬是 b，單向 cut capacity 是 $64b$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.8 Multiprocessor network topologies （續）

Central crossbar 不是 K_{16}。N-port crossbar 的 endpoint 通常各接一個 switch port，內部有 $O(N^2)$ crosspoints；它的 endpoint degree、線路長度、仲裁成本與 cut bandwidth 必須用 switch 模型另算。

Total network bandwidth 是「每條 link 頻寬 × link 數」的上界；bisection bandwidth 則取所有 balanced cuts 的最小 cut bandwidth，較接近 worst-case pattern。兩者都不足以單獨預測 application latency；hotspot、routing、message size 與 flow control 都會改變結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.9 Multiprocessor benchmarks

Parallel benchmark 要固定程式、data set、validation、compiler flags 與 processor count。Weak scaling benchmark 可讓 problem size grow，但要記錄 growth rule。

Linpack 是 dense linear algebra 與弱 scaling 常見例；SPECrate 適合 independent job throughput；SPLASH 類 benchmark 研究 shared-memory application patterns。

Peak performance 與 measured application performance 常不同。Memory bandwidth、load imbalance、communication 與 synchronization 可能占主要時間。

對 parallel systems 要看 per-core efficiency、scaling curve、tail latency、energy 與 fault tolerance，不只 single FLOPS。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.10 Roofline 的推導與數值工作點

令某 kernel 一個 phase 執行 F 次 floating-point operations，跨 memory 介面實際搬移 Q bytes。Arithmetic intensity 定義為

$$I = \frac{F}{Q} \quad (\mathrm{FLOP/byte}).$$

若 compute 的 sustained peak 是 P_C，該 memory 介面的持續 bandwidth 是 B，同一 phase 的 compute 與 memory times 分別是

$$T_{\mathrm{compute}} = \frac{F}{P_C}, \qquad T_{\mathrm{memory}} = \frac{Q}{B} = \frac{F}{IB}.$$

若 compute 與 traffic 可以完全重疊，elapsed time 不可能小於兩者的最大值：

$$T \geq \max\left(\frac{F}{P_C}, \frac{Q}{B}\right),$$

所以 throughput 上界為

$$P \leq \frac{F}{T} \leq \min(P_C, IB).$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.10 Roofline 的推導與數值工作點 （續）

令 $IB = P_C$ 可求兩條 roof 的折點：

$$I_{\text{ridge}} = \frac{P_C}{B} \quad (\text{FLOP/byte}).$$

$I < I_{\text{ridge}}$ 是 memory-bound；$I > I_{\text{ridge}}$ 是 compute-bound。圖的上界不代表實測一定達到，optimization 仍需處理 dependencies、instruction mix、load imbalance 與 memory transaction efficiency。

單層工作點

取 $P_C = 100$ GFLOP/s、$B = 50$ GB/s，ridge 是

$$I_{\text{ridge}} = 100/50 = 2.$$

Kernel A：$I = 0.5$，memory roof $IB = 25$，compute roof 100：

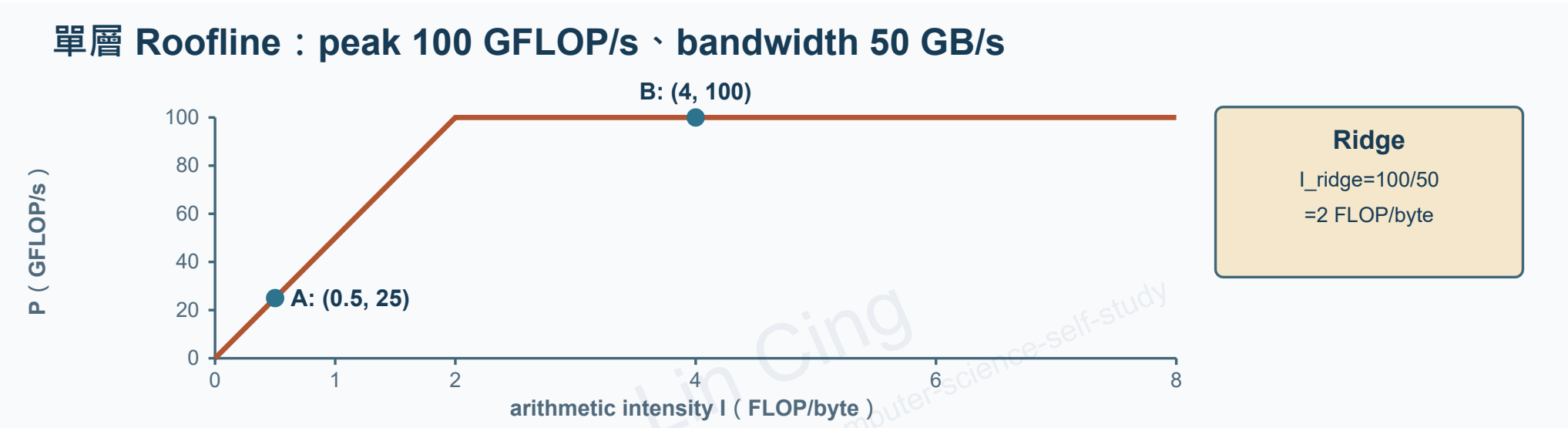
$$P_A \le \min(100, 25) = 25 \ \text{GFLOP/s}.$$

Kernel B：$I = 4$，memory roof $IB = 200$，但 compute cap 100：

$$P_B \le \min(100, 200) = 100 \ \text{GFLOP/s}.$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.10 Roofline 的推導與數值工作點 （續）

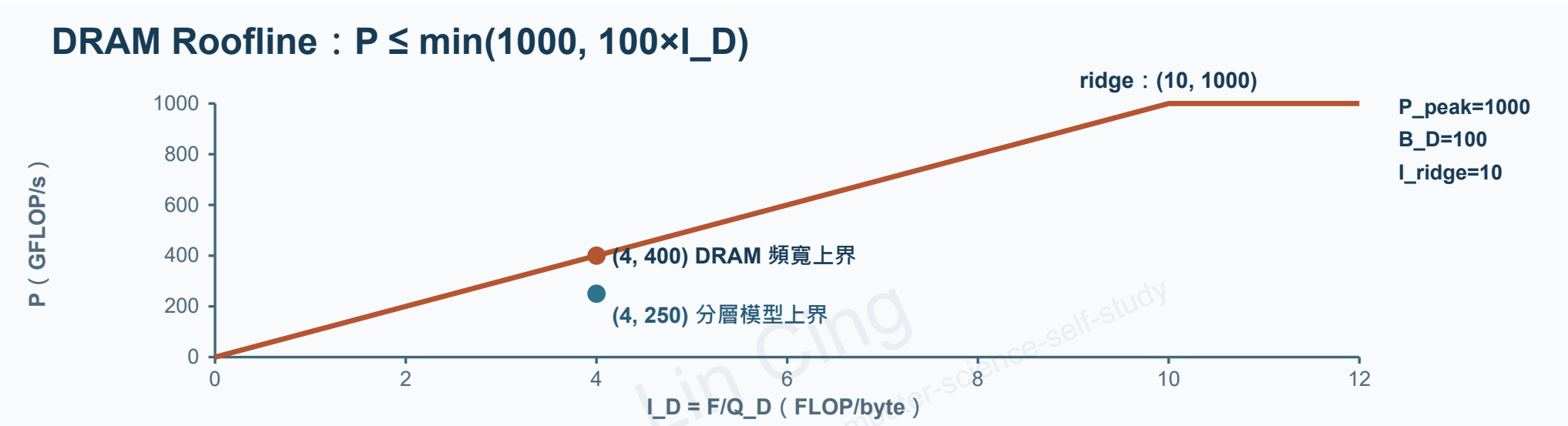


階層 Roofline 的同一組數字

固定一階段 $F = 10^9$ FLOPs、$P_C = 1000$ GFLOP/s。DRAM 的 $Q_D = 0.25 \times 10^9$ bytes、$B_D = 100$ GB/s；shared memory 的 $Q_S = 4 \times 10^9$ bytes、$B_S = 1000$ GB/s。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.10 Roofline 的推導與數值工作點 （續）

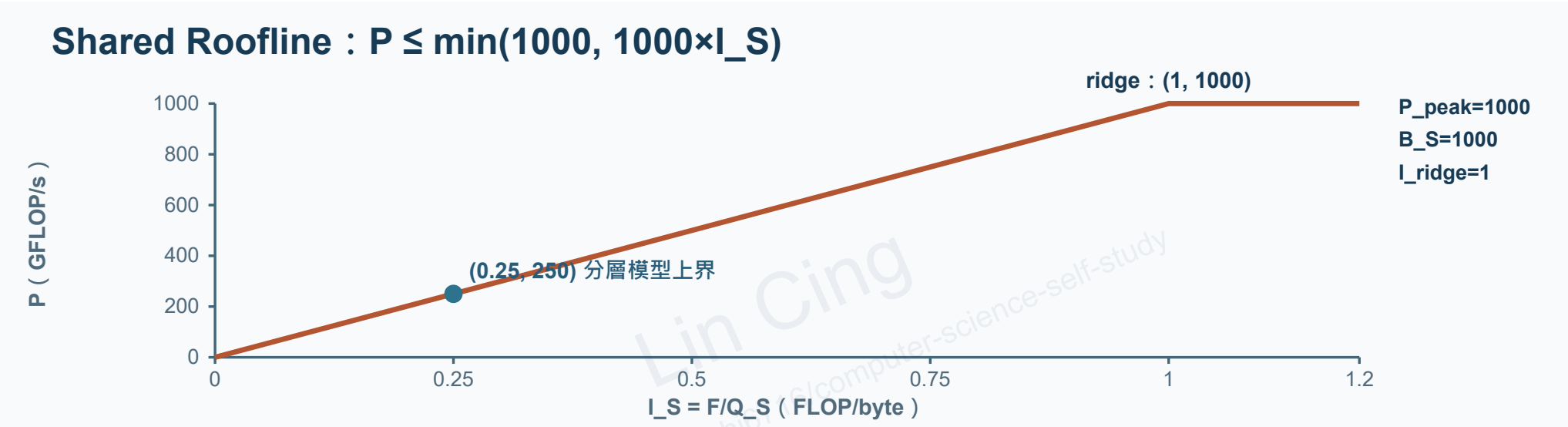


DRAM 層：

$$I_D = \frac{10^9}{0.25 \times 10^9} = 4, \qquad \min(1000, 100 \cdot 4) = 400 \ \text{GFLOP/s}.$$

7.10 Roofline 的推導與數值工作點 （續）

shared 層：



$$I_S = \frac{10^9}{4 \times 10^9} = 0.25, \qquad \min(1000, 1000 \cdot 0.25) = 250 \text{ GFLOP/s}.$$

把三層限制合併：

$$P \le \min(1000, 400, 250) = 250 \text{ GFLOP/s}.$$

7.10 Roofline 的推導與數值工作點 （續）

或等價地比較時間：

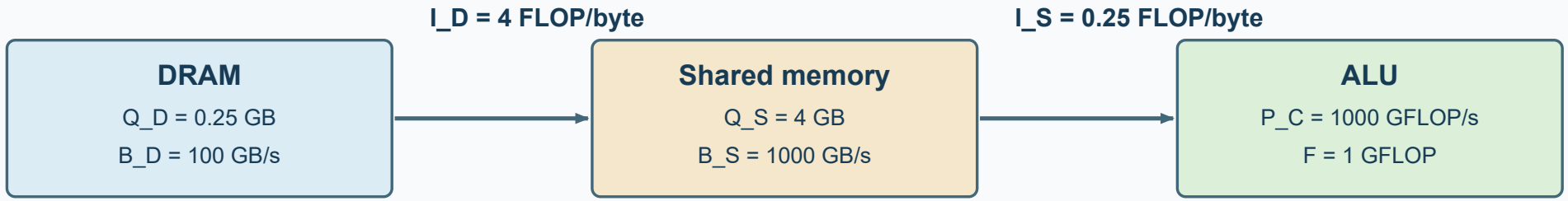
$$T \geq \max\left(\frac{10^9}{10^{12}}, \frac{0.25 \times 10^9}{100 \times 10^9}, \frac{4 \times 10^9}{1000 \times 10^9}\right) = \max(0.001, 0.0025, 0.004) = 0.004\ \text{s}.$$

也就是 time lower bound 是 4 ms。

因此在這個明定模型中，DRAM 的工作點是 $(4, 400)$，但 shared 層把跨層上界壓到 $(4, 250)$。250 是模型上界，不是 measured performance；shared roof 本身仍以 1000 GFLOP/s 為 plateau，不能把 250 誤畫成整個 plateau。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一階段的 DRAM、shared 與 compute 資源時間



T_compute = 1 ms；T_DRAM = 2.5 ms；T_shared = 4 ms。

T ≥ max(1, 2.5, 4) = 4 ms；P ≤ min(1000, 400, 250) = 250 GFLOP/s。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

若資料已被 cache 命中，就必須用該 cache 介面的 bytes 與 bandwidth 重算，不能把 DRAM 的 Q_D 和 cache bandwidth 混用。多層 roof 的圖可以分開畫，但最終 bound 要取同一 phase 各資源的最小值。

7.11 Real multicore comparison

本章案例以 four multicore systems 與 sparse matrix、structured grid kernels 展示 roofline 的實用方式。不同 systems 的 peak FLOPS、memory bandwidth、cache/TLB 與 programming model 不同；measured point 與 roof 的距離指出 optimization headroom。

Sparse matrix 的 indirect access 與 structured grid 的 stencil access 有不同 locality，因此即使 arithmetic intensity 相近，pipeline/coalescing/compiler quality 仍造成差異。

歷史結論不能當成今日 benchmark；本章保留 metric 與 interpretation 方法。

圖上較高 peak 且較慢 measured kernel 很常見；peak 是外框，不是可交付應用效能。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.12 Parallel fallacies

「Amdahl's law 不適用」常源於拿 weak scaling 去反駁 strong-scaling claim；fixed-work model 沒有規定無限 speedup forever。

「peak FLOPS 決定最快系統」忽略 memory bound、load imbalance、communication 與 software stack。

「more threads 必然更快」忽略 synchronization, oversubscription, cache thrashing 與 power。

「coherent shared memory 讓 race-free」不成立；data races 仍需要 synchronization 與 memory model。

前述80/20不等成本例否定「operation share等於time share」；單一worker分到5%的例把100核speedup壓到19.63；publication trace則指出coherence無法替代跨位置同步。這些都是明定模型的反例，不冒充實機量測。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.13 Concluding remarks

Parallel hardware 提供 resources；parallel software 把它們轉成具體 speedup。Strong/weak scaling、load balance、communication、synchronization、coherence 與 benchmark 設計共同決定結果。

Shared-memory 與 message-passing 是 different address-space models。Multithreading、SIMD、GPU 與 clusters 各以不同成本隱藏 latency 或增加 throughput。

Roofline 將 compute roof 與 memory roof 結合，能指出 kernel 是 compute-bound 或 memory-bound；它只給 upper bound，實測仍需 performance counter 與 end-to-end benchmark。

若 DRAM roof 是最低上限，且工作集、資料流量與其他限制固定，提高 compute peak 不會提高該上限。可用相同完整工作階段的 DRAM bytes、持續 bandwidth 與執行時間檢查此預測；若資料重用改變，須先重算 intensity。

章末驗收

應能：

1. 給定 parallel fraction、worker count 與 overhead，計算 speedup/efficiency。
2. 分開 strong/weak scaling 的 problem-size 假設與 conclusion。
3. 說明 SMP、NUMA、cluster 的 address space 與 communication cost。
4. 比較 fine/coarse multithreading 與 SMT 的 stall hiding。
5. 計算 mesh 的 degree、diameter、bisection links。
6. 從 peak FLOPS、DRAM bandwidth 與 arithmetic intensity 判定 roofline bound。

先完成本章獨立練習題 PDF，再由詳解 PDF 檢查假設與推導。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.14 平行架構的歷史視角

向量處理器用少量vector instructions描述大量規則運算，攤薄fetch與控制成本；shared-memory multiprocessors讓既有load/store程式模型延伸到多個processors，但需coherence及同步。Clusters以獨立節點和明確訊息傳遞換取可擴充資源與失效邊界，通信成本則直接進入演算法設計。

晶片整合增加後，multicore、SMT與GPU分別利用task、thread與data parallelism。它們沒有消除serial critical path，也沒有讓所有通信變成本地。比較歷史設計時應固定問題規模、數值驗證、記憶體容量、通信模式及power budget，才能分辨技術進步與工作負載改變。

完全連接graph與crossbar不同。K16每個endpoint連15條直接links，balanced cut有8×8=64條links；16-port crossbar通常每endpoint只連一個switch port，內部有$O(N^2)$crosspoints，其吞吐還受endpoint port rate限制，不能把兩者degree與cut-link數混成同一表。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料

Patterson 與 Hennessy，*Computer Organization and Design: The Hardware/Software Interface*，Revised Fourth Edition，7.2、7.8–7.10，印刷頁 635–674（PDF 頁 662–701）。

Williams、Waterman 與 Patterson，〈Roofline: An Insightful Visual Performance Model for Multicore Architectures〉，*Communications of the ACM* 52(4), 2009，DOI。

分層流量定義另參照 NERSC Roofline 文件；每個層級以該介面的 bytes 計算自己的 intensity。本文數字是明定模型的代入與推導。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study